

Evoluzione del progetto Java Multithreading

Percorso progressivo: da **Thread** a **CompletableFuture**

Il progetto è stato costruito secondo un approccio progressivo: partire dai concetti fondamentali della programmazione concorrente, introdurre gradualmente gli strumenti della libreria Java e arrivare infine a una soluzione concreta e moderna per l'esecuzione asincrona di operazioni di I/O.

L'esempio applicativo utilizzato come filo conduttore è il **download concorrente di pagine web**, con salvataggio del contenuto su file e produzione di un report finale.

L'evoluzione può essere sintetizzata così:

```
Thread
↓
Runnable
↓
Thread concorrenti + join()
↓
Download concorrente di pagine web
↓
ExecutorService / Thread Pool
↓
Callable + Future
↓
CompletionService
↓
CompletableFuture
↓
Timeout + gestione errori + report finale
```

lez01 — Introduzione ai Thread

Obiettivo

La prima lezione introduce il concetto fondamentale di **thread di esecuzione** e mostra come Java permetta di creare ed eseguire più flussi di lavoro all'interno della stessa applicazione.

Il punto di partenza è il thread principale:

```
JVM
├─ main thread
│   └─ main()
```

Il metodo `main()` viene eseguito dal **main thread**, creato dalla JVM all'avvio dell'applicazione.

Concetti introdotti

`Thread.currentThread()`

Permette di ottenere il thread che sta eseguendo il codice in quel momento.

È importante perché le operazioni come:

- `sleep()`
- `getName()`
- `getPriority()`
- `isAlive()`

si riferiscono al thread corrente o a uno specifico oggetto `Thread`.

Nome del thread

Ogni thread possiede un nome.

Il nome può essere modificato con:

```
setName()
```

e recuperato con:

```
getName()
```

Il nome è particolarmente utile durante il debugging e nei programmi concorrenti, perché permette di capire quale thread sta eseguendo una determinata operazione.

Priorità

Java assegna ai thread una priorità compresa tra:

```
1 → priorità minima  
5 → priorità normale  
10 → priorità massima
```

La priorità rappresenta però una **preferenza per lo scheduler**, non una garanzia sull'ordine di esecuzione.

Un thread con priorità maggiore non significa necessariamente che terminerà prima.

`sleep()`

`Thread.sleep()` sospende temporaneamente **il thread corrente**.

È importante sottolineare che:

```
sleep() non ferma la JVM  
sleep() non ferma tutti i thread
```

Sospende solamente il thread che sta eseguendo quella chiamata.

`start()` e `run()`

Uno dei concetti fondamentali dell'intero percorso è la differenza tra:

```
start()
```

e:

```
run()
```

`start()` chiede alla JVM di avviare un nuovo thread di esecuzione.

Chiamare direttamente `run()` è invece una normale chiamata di metodo e **non crea un nuovo thread**.

Thread daemon

Viene introdotto anche il concetto di **daemon thread**.

Un daemon thread è un thread di servizio che non impedisce alla JVM di terminare.

La distinzione importante è:

```
User Thread  
  ↓  
può mantenere viva la JVM  
  
Daemon Thread  
  ↓  
non mantiene viva la JVM
```

Essere daemon e avere una determinata priorità sono due caratteristiche indipendenti.

Prima implementazione

Il primo esempio utilizza una classe che estende:

Thread

e ridefinisce:

run()

L'esempio serve principalmente a osservare:

- nome del thread;
- priorità;
- stato;
- `isAlive()`;
- `sleep()`;
- daemon;
- avvio con `start()`.

Obiettivo didattico raggiunto

Alla fine di `lez01` lo studente comprende che un programma Java non deve necessariamente essere costituito da un unico flusso sequenziale.

È possibile avere:

```
main
├── Thread 1
├── Thread 2
└── Thread 3
```

con più attività eseguite concorrentemente.

lez02 — Runnable

Obiettivo

La seconda lezione introduce una soluzione più flessibile rispetto all'ereditarietà da `Thread`:

Runnable

Il concetto fondamentale è la separazione tra:

attività da eseguire

e:

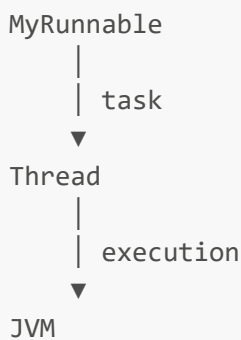
meccanismo che esegue l'attività

Runnable come task

Una classe che implementa `Runnable` descrive **cosa deve essere eseguito**.

Il `Thread` rappresenta invece il **thread di esecuzione**.

Schema:



Questa separazione è molto importante perché sarà alla base delle soluzioni successive.

Vantaggi rispetto a `extends Thread`

Con:

`extends Thread`

la classe diventa contemporaneamente:

- un oggetto applicativo;
- un thread.

Con:

`implements Runnable`

la classe rappresenta invece solamente il lavoro da eseguire.

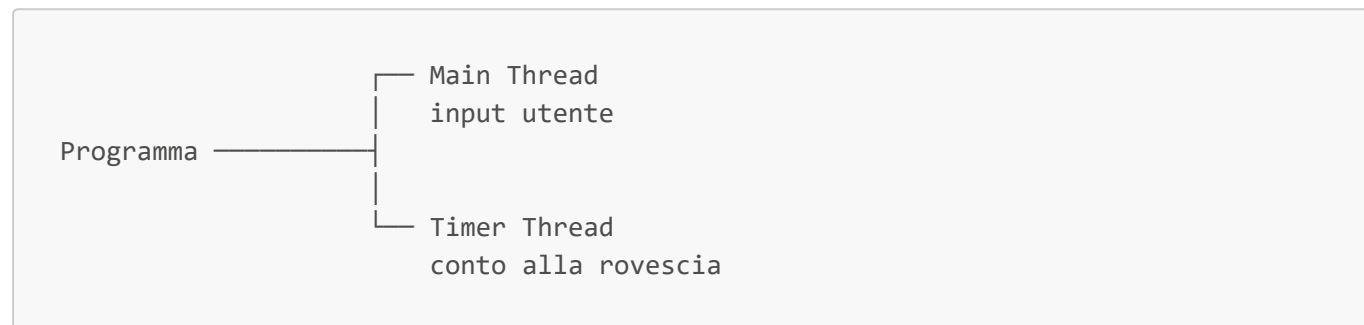
Il thread può quindi essere creato separatamente.

Esempio del timer

Il progetto utilizza un semplice timer come applicazione concreta.

Il `main thread` chiede all'utente di inserire il proprio nome mentre un secondo thread esegue il conto alla rovescia.

Schema:



Questo esempio rende evidente uno dei principali vantaggi del multithreading:

mentre un thread è impegnato in un'operazione, un altro può continuare a lavorare.

InterruptedException

`Thread.sleep()` può essere interrotto.

Per questo motivo Java richiede la gestione di:

`InterruptedException`

L'interruzione rappresenta un meccanismo importante per comunicare a un thread che dovrebbe interrompere la propria attività.

Errore didattico importante: `System.exit()`

Nel primo esempio il timer utilizzava `System.exit(0)` alla fine del conto alla rovescia.

Questo è stato evidenziato come comportamento da evitare in una soluzione ben strutturata.

`System.exit(0)` infatti:

termina l'intera JVM

e non solamente il thread che lo esegue.

È preferibile lasciare terminare naturalmente il metodo `run()`.

Obiettivo didattico raggiunto

Lo studente comprende la differenza tra:

Task

e:

Thread

Questo concetto sarà fondamentale quando verranno introdotti gli **ExecutorService**.

lez03 — Thread concorrenti e `join()`

Obiettivo

La terza lezione mostra cosa accade quando vengono creati realmente più task concorrenti.

L'esempio utilizza due istanze dello stesso `Runnable`:

PING
PONG

Ogni task stampa il proprio testo più volte con una pausa tra una stampa e l'altra.

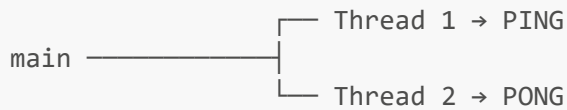
Due task indipendenti

Il programma crea:

```
MyRunnable("PING")  
MyRunnable("PONG")
```

e successivamente due `Thread`.

Schema:



I due thread vengono avviati con `start()`.

Ordine di esecuzione

L'ordine delle stampe non è deterministico.

È quindi possibile ottenere, per esempio:

```
PING
PONG
PING
PONG
```

ma anche:

```
PONG
PING
PING
PONG
```

oppure altre combinazioni.

Questo introduce concretamente il concetto di **concorrenza**.

Il programmatore definisce i task, ma non controlla direttamente ogni singolo istante nel quale lo scheduler eseguirà ciascun thread.

`join()`

A questo punto viene introdotto:

```
join()
```

`join()` permette a un thread di attendere la terminazione di un altro thread.

Nel nostro esempio:

```
main
|
├─ avvia PING
```

```
|— avvia PONG  
|— join(PING)  
|— join(PONG)  
|— GAME OVER
```

Il punto fondamentale è:

`join()` blocca il thread corrente fino alla terminazione del thread sul quale viene invocato.

Non blocca il thread target.

Perché è importante?

Senza `join()`, il `main` potrebbe raggiungere:

```
GAME OVER!
```

prima che PING e PONG abbiano terminato.

Con `join()` si può invece esprimere una relazione di dipendenza:

```
esegui i task  
  ↓  
attendi che terminino  
  ↓  
continua
```

Risultato didattico

A questo punto lo studente conosce:

- `Thread`;
- `Runnable`;
- `start()`;
- `run()`;
- `sleep()`;
- `InterruptedException`;
- daemon thread;
- `join()`;
- esecuzione concorrente.

È quindi possibile passare da esempi puramente didattici a un'applicazione concreta.

lez04 — Downloader web concorrente

Obiettivo

La quarta lezione introduce un problema reale:

scaricare contemporaneamente il contenuto di più pagine web.

Questo permette di applicare i concetti precedenti a un'operazione di I/O.

Scenario

Il programma dispone di una lista di URL:

```
URL 1
URL 2
URL 3
URL 4
...
```

Per ogni URL deve:

1. effettuare una richiesta HTTP;
2. ricevere il contenuto;
3. verificare il risultato;
4. salvare il contenuto in un file.

HttpClient

Per le richieste HTTP viene utilizzato:

```
java.net.http.HttpClient
```

disponibile a partire da Java 11.

Il flusso concettuale è:

```
URL
↓
HttpRequest
↓
HttpClient
↓
HttpResponse
↓
contenuto
```

↓
file

Prima soluzione: un Thread per ogni pagina

La prima implementazione utilizza ancora direttamente **Thread**.

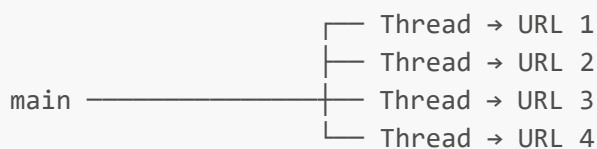
Per ogni URL viene creato:

WebPageDownloader

che implementa **Runnable**.

Successivamente viene creato un **Thread** e viene eseguito con **start()**.

Schema:



Vantaggi

Il programma permette di scaricare più pagine contemporaneamente.

Se un download è lento, gli altri thread possono continuare.

Limite della soluzione

Creare manualmente un thread per ogni attività non è una soluzione scalabile.

Con:

10 URL

il problema è ancora trascurabile.

Con:

1.000 URL

creare 1.000 thread non è una buona strategia.

Occorre quindi separare ulteriormente:

```
task
```

da:

```
gestione dei thread
```

Questa esigenza introduce `ExecutorService`.

lez05 — `ExecutorService`, `Callable` e `Future`

Obiettivo

La quinta lezione sostituisce la gestione manuale dei thread con un **Thread Pool**.

Il concetto fondamentale diventa:

```
non creo un thread per ogni task
```

ma:

```
creo un insieme limitato di thread  
e assegno loro i task da eseguire.
```

`ExecutorService`

L'`ExecutorService` gestisce un insieme di thread riutilizzabili.

Esempio concettuale:

```
Task 1 ┌  
Task 2 ┌  
Task 3 ┌─> Thread Pool  
Task 4 ┌  
Task 5 ┌
```

con, ad esempio:

```
3 thread
```

Il pool può quindi eseguire al massimo tre task contemporaneamente.

Gli altri rimangono in coda.

`newFixedThreadPool()`

La soluzione utilizza un pool a numero fisso di thread.

Il numero di thread viene scelto in base al tipo di lavoro e alle caratteristiche dell'applicazione.

Il vantaggio rispetto alla soluzione precedente è che i thread vengono **riutilizzati**.

`submit()`

I task vengono inviati al pool tramite:

```
submit()
```

L'`ExecutorService` decide quale thread libero dovrà eseguire ciascun task.

`shutdown()`

Quando non devono essere inviati altri task viene invocato:

```
shutdown()
```

Questo non interrompe immediatamente i task già inviati.

Indica che:

```
non verranno accettati nuovi task
```

mentre quelli già presenti possono terminare.

Dal `Runnable` al `Callable`

A questo punto emerge una nuova esigenza.

Il downloader non deve soltanto eseguire un'operazione:

```
download
```

ma dovrebbe anche restituire un risultato:

```
OK / ERROR  
URL  
file  
messaggio
```

Per questo viene introdotto:

```
Callable<T>
```

Callable<DownloadResult>

Il task diventa quindi un:

```
Callable<DownloadResult>
```

e il metodo principale non è più:

```
run()
```

ma:

```
call()
```

La differenza fondamentale è:

```
Runnable  
  run()  
  nessun valore restituito  
  
Callable<T>  
  call()  
  restituisce T
```

DownloadResult

Il risultato viene rappresentato con un **record**.

Concettualmente contiene:

```
URL
nome file
successo/fallimento
messaggio
```

Il **record** è particolarmente adatto perché il risultato è principalmente un contenitore immutabile di dati.

Future

Quando un **Callable** viene inviato all'**ExecutorService**, viene restituito un:

```
Future<DownloadResult>
```

Il **Future** rappresenta il risultato che sarà disponibile successivamente.

Schema:

```
submit()
  ↓
Future
  ↓
task in esecuzione
  ↓
DownloadResult
```

get()

Il risultato viene recuperato con:

```
get()
```

Il problema è che **get()** è bloccante.

Se il risultato non è ancora disponibile, il thread chiamante rimane in attesa.

Evoluzione concettuale

Si passa quindi da:

Thread

a:

ExecutorService

e da:

Runnable

a:

Callable<T>

ottenendo:

```
Callable<T>
  ↓
ExecutorService
  ↓
Future<T>
  ↓
get()
  ↓
risultato
```

lez06 — ExecutorCompletionService

Problema di Future.get()

La soluzione precedente presenta un limite.

Supponiamo di avere:

```
Future 1 → download lento
Future 2 → download veloce
Future 3 → download velocissimo
```

Se il programma esegue:

```
future1.get()  
future2.get()  
future3.get()
```

deve aspettare il primo risultato prima di poter elaborare gli altri.

Anche se **Future 2** e **Future 3** sono già terminati, il programma sta aspettando **Future 1**.

ExecutorCompletionService

Per risolvere questo problema viene introdotto:

```
ExecutorCompletionService
```

Il suo scopo è permettere di recuperare i risultati **nell'ordine in cui i task terminano**.

Schema:

```
Task 1 ————— 10 s  
Task 2 ————— 3 s  
Task 3 — 1 s  
Task 4 ————— 5 s
```

ordine di completamento:

```
Task 3  
Task 2  
Task 4  
Task 1
```

Il programma può quindi elaborare i risultati nello stesso ordine:

```
3 → 2 → 4 → 1
```

anziché:

```
1 → 2 → 3 → 4
```

submit()

I task vengono inviati al `CompletionService`.

Internamente vengono eseguiti dall'`ExecutorService`.

Il vantaggio è che il `CompletionService` mantiene una coda dei task completati.

`take()`

Il metodo:

```
take()
```

restituisce il prossimo `Future` disponibile.

Se nessun task è ancora terminato, `take()` attende.

Quindi:

```
take()
```

è un'operazione bloccante.

`poll()`

In alternativa:

```
poll()
```

non attende.

Se non c'è ancora nessun risultato disponibile restituisce:

```
null
```

La distinzione è:

```
take()
    attende

poll()
    controlla senza attendere
```

Vantaggio rispetto a `Future.get()`

Con `CompletionService` il programma può reagire immediatamente ai task che terminano.

Questo è particolarmente utile quando i task hanno tempi di esecuzione molto diversi.

Gestione dell'interruzione

Durante l'uso di `take()` viene nuovamente incontrata:

```
InterruptedException
```

In una gestione corretta dell'interruzione è buona pratica ripristinare lo stato di interrupt del thread tramite:

```
Thread.currentThread().interrupt()
```

Risultato della lezione

A questo punto abbiamo una soluzione più efficiente nella gestione dei risultati:

```
ExecutorService
    ↓
CompletionService
    ↓
task completati
    ↓
risultati nell'ordine di completamento
```

Ma esiste ancora un ulteriore livello di astrazione, particolarmente interessante in Java moderno:

```
CompletableFuture
```

lez07 — `CompletableFuture`: programmazione asincrona

Obiettivo

L'ultima lezione porta il progetto verso un modello di programmazione **asincrona e composizionale**.

L'obiettivo non è più soltanto:

```
eseguire task concorrenti
```

ma costruire una vera e propria:

```
pipeline asincrona
```

sendAsync()

Il downloader utilizza il supporto asincrono di `HttpClient`:

```
sendAsync()
```

La richiesta HTTP non blocca il thread chiamante.

Viene restituito un:

```
CompletableFuture<HttpResponse<String>>
```

che rappresenta un risultato che sarà disponibile in futuro.

Schema:

```
main
|
|— avvia richiesta 1 → CompletableFuture
|
|— avvia richiesta 2 → CompletableFuture
|
|— avvia richiesta 3 → CompletableFuture
|
|— continua
```

thenApply()

Una volta ricevuta la risposta HTTP, occorre trasformarla in un risultato applicativo.

`thenApply()` permette di descrivere questa trasformazione:

```
HttpResponse  
  ↓  
DownloadResult
```

È quindi una trasformazione:

```
A → B
```

thenAccept()

Quando il risultato deve essere semplicemente consumato, senza produrre un nuovo risultato, si utilizza:

```
thenAccept()
```

Esempio concettuale:

```
DownloadResult  
  ↓  
stampa risultato
```

thenRun()

`thenRun()` esegue un'azione al termine del precedente stage senza ricevere il risultato e senza produrne uno nuovo.

È utile, per esempio, per eseguire un'azione finale.

Gestione degli errori

Una pipeline asincrona deve prevedere anche il fallimento delle operazioni.

Viene quindi introdotto:

```
exceptionally()
```

che permette di trasformare un completamento eccezionale in un risultato gestibile.

Nel nostro progetto questo permette di trasformare un errore di download in un:

```
DownloadResult
```

con:

```
success = false
```

handle()

Viene inoltre introdotto il concetto di:

```
handle()
```

che permette di gestire insieme:

```
risultato
```

e:

```
eccezione
```

È particolarmente utile quando la pipeline deve produrre comunque un risultato finale.

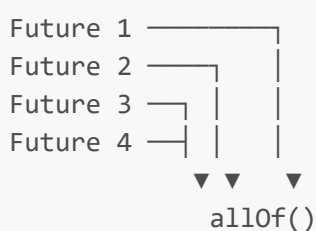
allOf() — attendere tutti i download

Dopo aver creato una lista di `CompletableFuture`, è necessario sapere quando tutti i download sono terminati.

Viene utilizzato:

```
CompletableFuture.allOf(...)
```

Concettualmente:



↓
tutti completati

`allOf()` produce un `CompletableFuture<Void>` che termina quando tutti i future forniti sono completati.

Il risultato dei singoli download deve quindi essere recuperato dai singoli `CompletableFuture`.

Timeout

Una pipeline reale deve gestire anche il caso in cui un server non risponda.

Viene quindi introdotto:

```
orTimeout()
```

Nel progetto viene impostato un timeout di:

```
10 secondi
```

Se il download non termina entro il tempo stabilito, il `CompletableFuture` viene completato eccezionalmente.

La gestione tramite `exceptionally()` permette di trasformare anche il timeout in un normale:

```
DownloadResult
```

con esito negativo.

Schema:



In questo modo il report finale può trattare nello stesso modo:

- errore HTTP;
- errore di scrittura del file;
- timeout;
- altre eccezioni.

Report finale

Una volta terminati tutti i download viene prodotto un report.

Il report contiene almeno:

```
URL
file destinazione
esito
messaggio
```

e un riepilogo:

```
Totale
Completati
Falliti
```

stampaReport()

La logica di generazione del report viene separata dal `main()` in un metodo dedicato.

Questo permette di mantenere distinta:

```
logica di avvio
```

da:

```
logica di presentazione dei risultati
```

Il metodo analizza i `CompletableFuture` ormai completati e recupera i rispettivi `DownloadResult`.

Il ruolo dell'ultimo `join()`

Alla fine della pipeline viene utilizzato:

```
join()
```

ma con un significato diverso rispetto alla prima introduzione di `join()` in [lez03](#).

In [lez03](#)

`join()` viene utilizzato per attendere singoli thread:

```
main
  ↓
join(thread1)
  ↓
join(thread2)
```

In [lez07](#)

`join()` viene utilizzato una sola volta sulla pipeline complessiva:

```
allOf()
  ↓
thenRun(report)
  ↓
join()
```

In questo caso il `main` attende che sia completata l'intera elaborazione asincrona.

Non viene effettuato un `join()` separato per ogni download.

Confronto delle soluzioni

Il progetto mostra una vera evoluzione degli strumenti Java per la concorrenza.

Lezione	Tecnologia	Concetto principale
lez01	Thread	Creazione e gestione di un thread
lez02	Runnable	Separazione tra task e thread
lez03	Thread + join()	Coordinamento tra thread
lez04	HttpClient + Thread	Applicazione concreta: download concorrenti
lez05	ExecutorService	Thread Pool e gestione dei task
lez05	Callable + Future	Task con valore restituito

Lezione	Tecnologia	Concetto principale
lez06	CompletionService	Risultati nell'ordine di completamento
lez07	CompletableFuture	Programmazione asincrona composizionale
lez07	allOf()	Coordinamento di più operazioni asincrone
lez07	orTimeout()	Gestione dei timeout
lez07	exceptionally()	Gestione degli errori
lez07	Report finale	Raccolta e presentazione dei risultati

Evoluzione architetturale

Il valore didattico del progetto non è soltanto conoscere singole API, ma osservare come cambia progressivamente l'architettura.

Fase 1 — Thread manuali

```
Task
↓
Thread
↓
start()
```

Il programmatore gestisce direttamente i thread.

Fase 2 — Runnable

```
Task
↓
Runnable
↓
Thread
↓
start()
```

Il task viene separato dal meccanismo di esecuzione.

Fase 3 — ExecutorService

```
Task
  ↓
ExecutorService
  ↓
Thread Pool
```

La gestione dei thread viene delegata all'executor.

Fase 4 — Callable + Future

```
Callable<T>
  ↓
ExecutorService
  ↓
Future<T>
  ↓
risultato
```

Il task può produrre un risultato.

Fase 5 — CompletionService

```
Callable<T>
  ↓
CompletionService
  ↓
risultati completati
  ↓
ordine di completamento
```

Il programma può reagire ai risultati man mano che diventano disponibili.

Fase 6 — CompletableFuture

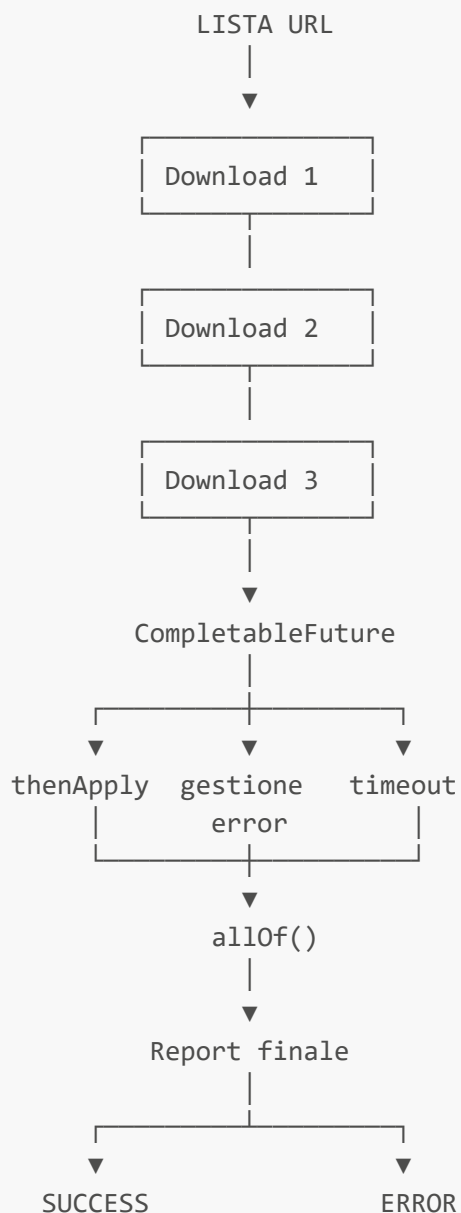
```
operazione asincrona
  ↓
CompletableFuture
  ↓
thenApply()
  ↓
thenAccept()
  ↓
```

```
allOf()  
  ↓  
report
```

La concorrenza viene trasformata in una **pipeline di operazioni asincrone**.

Il progetto finale

Alla fine delle sette lezioni il downloader può essere rappresentato concettualmente in questo modo:



Obiettivi didattici complessivi

Al termine del percorso lo studente dovrebbe essere in grado di:

- comprendere il concetto di thread;
- distinguere processo, thread e task;
- creare thread tramite `Thread`;
- utilizzare `Runnable`;
- distinguere `start()` da `run()`;
- utilizzare `sleep()`;
- comprendere `InterruptedException`;
- utilizzare `join()`;
- comprendere la differenza tra user thread e daemon thread;
- utilizzare `ExecutorService`;
- comprendere il concetto di Thread Pool;
- utilizzare `Callable`;
- utilizzare `Future`;
- comprendere il comportamento bloccante di `get()`;
- utilizzare `ExecutorCompletionService`;
- distinguere `take()` da `poll()`;
- utilizzare `CompletableFuture`;
- costruire pipeline asincrone;
- utilizzare `thenApply()`, `thenAccept()` e `thenRun()`;
- combinare più operazioni con `allOf()`;
- gestire le eccezioni con `exceptionally()` e `handle()`;
- gestire timeout;
- raccogliere risultati asincroni;
- produrre un report finale.

Sintesi finale

Il progetto segue una progressione volutamente graduale:

```
lez01
Thread
  ↓
lez02
Runnable
  ↓
lez03
Concorrenza + join()
  ↓
lez04
Downloader web
  ↓
lez05
ExecutorService + Callable + Future
  ↓
lez06
CompletionService
  ↓
```

```
lez07
CompletableFuture
↓
Timeout + Error Handling + Report
```

Il punto fondamentale è che **non si introducono gli strumenti avanzati in modo isolato**.

Ogni nuova tecnologia nasce dalla necessità di risolvere un limite della precedente:

```
Thread manuali
↓
troppa gestione manuale
↓
ExecutorService

Runnable
↓
serve un risultato
↓
Callable + Future

Future.get()
↓
attesa nell'ordine di sottomissione
↓
CompletionService

ExecutorService / gestione esplicita
↓
necessità di una pipeline asincrona
↓
CompletableFuture

Operazioni asincrone
↓
necessità di coordinamento
↓
allOf()

Operazioni che possono non terminare
↓
necessità di controllo del tempo
↓
orTimeout()

Operazioni che possono fallire
↓
necessità di gestione uniforme
↓
exceptionally() / handle()

Tutti i task terminati
```

```
graph TD; A[↓] --> B[raccolta dei risultati]; B --> C[↓]; C --> D[report finale];
```

↓
raccolta dei risultati
↓
report finale

In questo modo il progetto diventa un percorso completo dalla **programmazione concorrente di base** alla **programmazione asincrona moderna in Java**.